



Presented by: Emmanuel Goldberg

Agenda

01

Understanding Sentiment Analysis

Overview of core concepts and various approaches.

03

Hybrid Model Application

Applying the hybrid approach to Reddit stock market sentiments.

05

Comparative Analysis

Comparing the performance of both approaches.

02

Comparing Models

Detailed comparison of NLTK+VADER and Transformer-based models.

04

Transformer Model Application

Applying transformers to Reddit stock market sentiments.

06

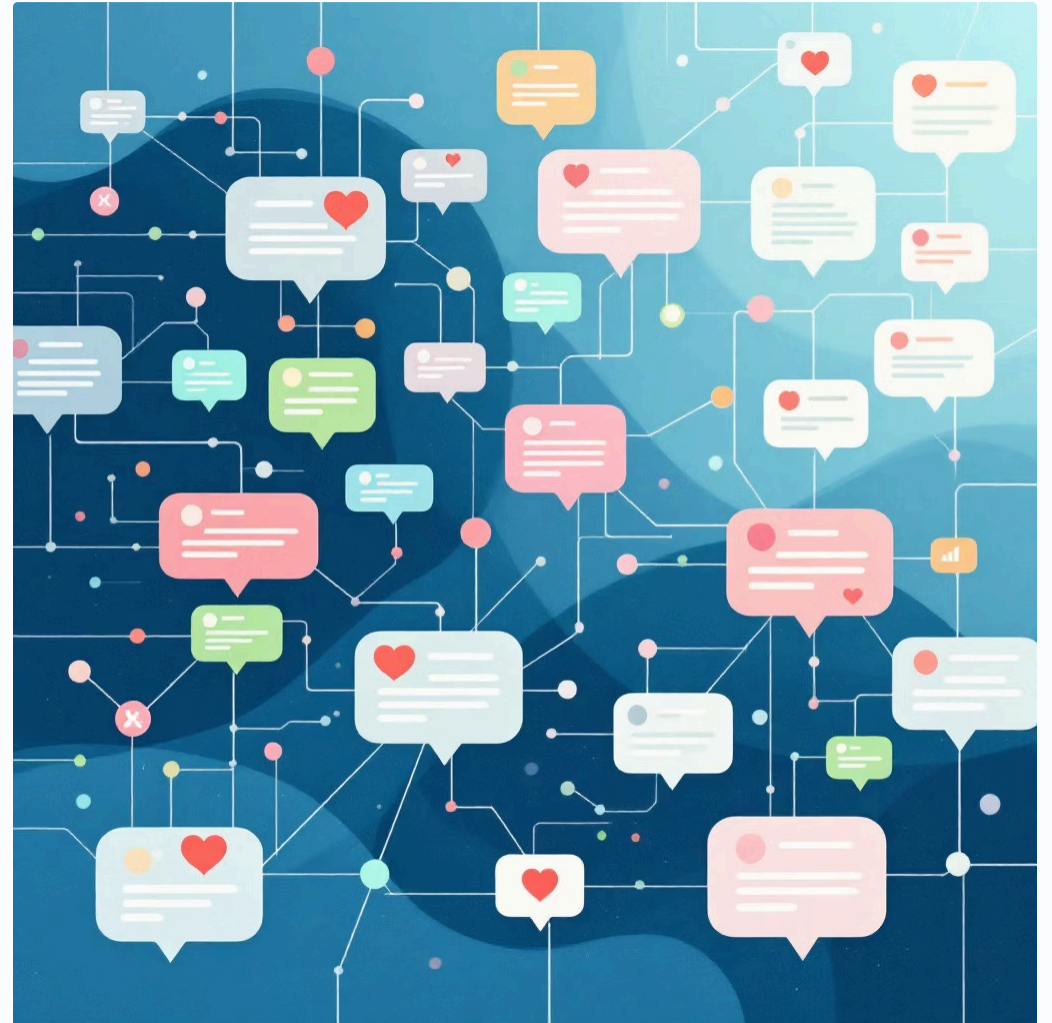
Conclusions and Takeaways

Summarizing key findings and future directions.

What is Social Media Sentiment Analysis?

Social media sentiment analysis categorizes chatter as "positive," "negative," or "neutral," consolidating it into a quantifiable metric. This allows you to track public perception of a brand over time and understand the emotions it evokes.

At its most basic, it flags customer feedback. Advanced Natural Language Processing (NLP) technology enables tools to accurately interpret emotional tones beyond simple classification.



Core Approaches to Sentiment Analysis

1

Rule-Based (Lexicon)

Utilizes pre-defined lexicons like VADER (Valence Aware Dictionary and sEntiment Reasoner) or SentiWordNet, assigning scores to words. Ideal for simple, fast analysis.

Example: "Great service, but slow delivery" → +0.3

2

Machine Learning

Learns patterns from labeled data using algorithms like Naïve Bayes or SVM (Support Vector Machine). Provides probabilistic sentiment.

Example: "Love the product, but packaging was bad" → 60% positive, 40% negative

3

Hybrid

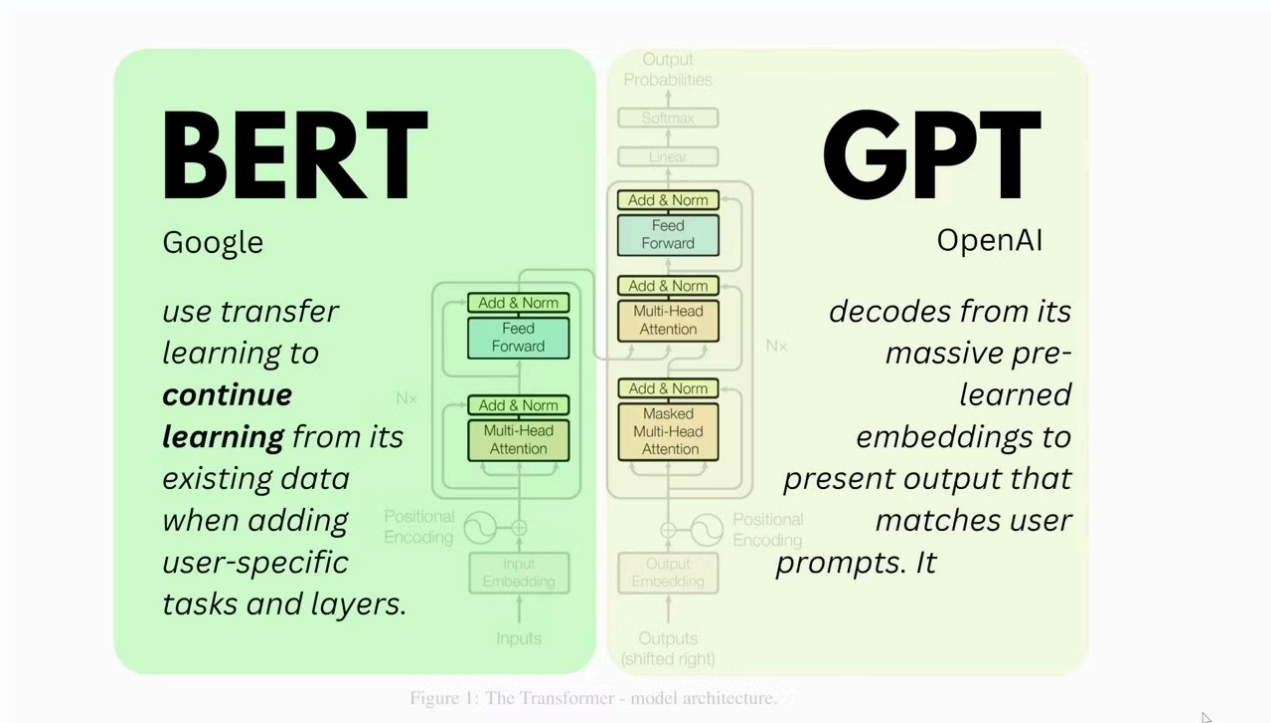
Combines rule-based methods with machine learning for enhanced accuracy and robustness.

4

Large Language Models (Transformers)

Fine-tuned models (BERT - Bidirectional Encoder Representations from Transformers, FinBERT, GPT-like) offer strong contextual understanding but are computationally intensive.

BERT (Bidirectional Encoder Representations from Transformers) - Introduced in 2018



Acts like an **Encoder - Decoder**

Bidirectional - Looks at words before and after

BERT is a stack of Transformer **encoder layers**.

Pre-training uses **Masked Language Modelling** (words randomly masked) and **Next Sentence Prediction** (forces to predict whether second sentence naturally follows first)

Final output are bidirectional contextualized embeddings.

Comparing Sentiment Analysis Models

Feature	NLTK + VADER	Transformer-Based (e.g., BERT, RoBERTa)
Technology	Rule-based, uses pre-defined lexicon and rules.	Deep learning, contextual embeddings, transformers.
Context Handling	Limited, relies on predefined rules.	High, understands word relationships and structure.
Accuracy	Moderate, best for short, informal text.	High, especially for nuanced/domain-specific text.
Speed	Very fast, suitable for real-time analysis.	Slower, computationally intensive.
Customization	Limited customization of rules/lexicons.	Fully customizable, fine-tune for specific tasks.
Resource Needs	Low, efficient on basic hardware.	High, requires GPUs/TPUs for training/inference.
Informal Text	Excellent, designed for social media slang.	Moderate, depends on training data.
Use Cases	Real-time social media, reviews, comments.	Domain-specific analysis (legal, medical, financial).

TF-IDF Vectorization for Numerical Representation

```
def compute_tfidf_sentiment(posts):  
    """  
    Input: list of text posts  
    Output: dict with bullish/bearish/neutral counts  
    """  
    if not posts:  
        return {"bullish": 0, "bearish": 0, "neutral": 0}  
  
    # Simple keyword-based sentiment for demonstration  
    bullish_keywords = ["buy", "bull", "long", "up"]  
    bearish_keywords = ["sell", "bear", "short", "down"]  
  
    processed_texts = [preprocess_text(p) for p in posts]  
  
    vectorizer = TfidfVectorizer()  
    X = vectorizer.fit_transform(processed_texts)  
  
    scores = {"bullish": 0, "bearish": 0, "neutral": 0}  
  
    for text in processed_texts:  
        text_tokens = text.split()  
        bullish_count = sum(1 for t in text_tokens if t in bullish_keywords)  
        bearish_count = sum(1 for t in text_tokens if t in bearish_keywords)  
        if bullish_count > bearish_count:  
            scores["bullish"] += 1  
        elif bearish_count > bullish_count:  
            scores["bearish"] += 1  
        else:  
            scores["neutral"] += 1  
  
    return scores
```

TfidfVectorizer()

Converts raw text into a **numerical sparse matrix** for ML models. TF-IDF (Term Frequency-Inverse Document Frequency) assigns higher weights to words frequent in one document but rare across others, making them more informative features.

fit_transform(processed_texts)

This method performs two crucial steps: it **fits** the vocabulary (all unique tokens) from `processed_texts` (e.g., preprocessed Reddit posts) and then **transforms** each document into a **TF-IDF weighted vector**. The resulting TF-IDF matrix X is then fed into a classifier.

Example: TF-IDF Sparse Matrix

Consider two documents from Reddit stock discussions after preprocessing:

- "aapl going moon"
- "tsla earnings bearish outlook"

The unique vocabulary would be: {"aapl", "going", "moon", "tsla", "earnings", "bearish", "outlook"}

```
X =  
[[0.707, 0.5, 0.5, 0, 0, 0, 0 ],  
 [0, 0, 0, 0.5, 0.5, 0.5, 0.5 ]]
```

The first row strongly weights "aapl," "going," and "moon," indicating their prominence in the first document. The second row highlights "tsla," "earnings," "bearish," and "outlook" for the second document. This matrix is then used for further classification.

VADER Sentiment Analysis Implementation

```
def compute_vader_sentiment(posts, min_length=5):
    """
    Compute VADER sentiment scores for a list of text posts with basic filtering.

    Parameters:
        posts (list of str): List of Reddit post contents (title + body).
        min_length (int): Minimum number of characters a post must have to be considered.

    Returns:
        dict: Normalized sentiment scores with keys:
            - 'bullish': proportion of positive posts
            - 'bearish': proportion of negative posts
            - 'neutral': proportion of neutral posts
    """
    scores = {'bullish': 0, 'bearish': 0, 'neutral': 0}

    # Filter out empty or very short posts to avoid noise
    filtered_posts = [p for p in posts if p and len(p.strip()) >= min_length]

    for post in filtered_posts:
        vs = analyzer.polarity_scores(post) # returns 'neg', 'neu', 'pos', 'compound'
        compound = vs['compound'] # range -1 to 1
        if compound >= 0.05:
            scores['bullish'] += 1
        elif compound <= -0.05:
            scores['bearish'] += 1
        else:
            scores['neutral'] += 1

    total = len(filtered_posts)
    if total > 0:
        # Normalize to proportions
        scores = {k: v / total for k, v in scores.items()}
    else:
        # Avoid division by zero
        scores = {'bullish': 0, 'bearish': 0, 'neutral': 0}

    return scores
```

`analyzer = SentimentIntensityAnalyzer()` creates a VADER object, loading a lexicon with pre-assigned sentiment values. It intelligently handles punctuation, capitalization, degree modifiers, and even slang/emojis.

`vs = analyzer.polarity_scores(post)` processes a text and returns a dictionary with negative, neutral, positive, and an overall compound score (normalized to -1 to +1).

`compound = vs['compound']` extracts this score. VADER's authors recommend thresholds: **compound** ≥ 0.05 for **positive (bullish)**, **compound** ≤ -0.05 for **negative (bearish)**, otherwise **neutral**. This maps continuous sentiment to discrete market categories.



Why VADER is Useful for Financial Sentiment

- **Quick, Rule-Based Sentiment**

VADER requires no prior training, making it highly efficient for rapid sentiment analysis.

- **Social Media Compatibility**

Its design handles informal language, all caps, exclamation marks, and slang effectively, which is crucial for social media data.

- **Aggregated Insights**

You can aggregate sentiment across numerous Reddit posts to determine percentage bullish vs. bearish, engagement-weighted scores, and sentiment ratios, providing a comprehensive market outlook.

Transformer-Based Sentiment Analysis with Hugging Face

```
from transformers import pipeline
# -----
# 📘 Hugging Face Sentiment Model
# -----
sentiment_model = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

def compute_transformer_sentiment(posts, min_length=5, neutral_threshold=0.65):
    """
    Compute sentiment scores using Hugging Face Transformers (DistilBERT SST-2).
    """
    scores = {'bullish': 0, 'bearish': 0, 'neutral': 0}
    filtered_posts = [p for p in posts if p and len(p.strip()) >= min_length]
    if not filtered_posts:
        return scores

    results = sentiment_model(filtered_posts, truncation=True)
    for res in results:
        label, conf = res['label'], res['score']
        if conf < neutral_threshold:
            scores['neutral'] += 1
        elif label == "POSITIVE":
            scores['bullish'] += 1
        elif label == "NEGATIVE":
            scores['bearish'] += 1

    total = len(filtered_posts)
    return {k: v / total for k, v in scores.items()}
```

The `pipeline("sentiment-analysis", model=...)` API from Hugging Face simplifies loading **pre-trained models** like `distilbert-base-uncased-finetuned-sst-2-english`. This model classifies text into "POSITIVE" or "NEGATIVE" with a confidence score.

The `compute_transformer_sentiment` function processes Reddit posts, initializing counters for **bullish**, **bearish**, and **neutral** sentiments. It filters out short posts and **batches** them for efficient inference using `truncation=True` to handle long inputs.

A **neutral threshold** can be applied: if the model's confidence score is **below 0.65**, the sentiment is categorized as neutral, preventing over-interpretation of weak predictions and providing a more robust sentiment classification.

Comparison with VADER

✅ **VADER** → Rule-based, fast, interpretable, good for slang/emojis.

• **Transformers (DistilBERT / FinBERT)** → Deep learning, captures **context**, more nuanced but heavier.

• **Market Mapping:**

• "POSITIVE" → **Bullish**

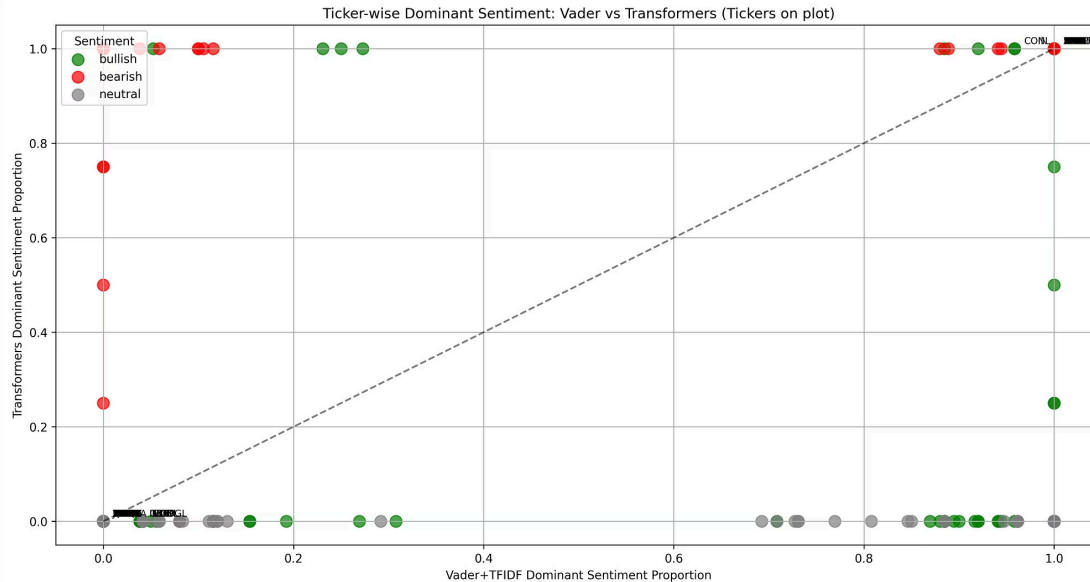
• "NEGATIVE" → **Bearish**

• Low confidence → **Neutral**

⚡ In the financial Reddit pipeline: VADER is the lightweight baseline.

Transformer is the domain-aware model (and ideally you'd swap distilbert-sst2 with FinBERT, trained on financial text).

Comparing reddit results:



Each point = one ticker, colored by sentiment.

X-axis = Vader proportion. Dominant = Bullish, Y-axis = Transformers proportion for that sentiment. Dominant = Bearish

Diagonal line = perfect agreement between models.

Points above diagonal → Transformers assigns higher proportion than Vader.

Points below diagonal → Vader assigns higher proportion than Transformers.

Key Takeaways

Hybrid Approach (VADER + TF-IDF)

Offers a fast, interpretable, and solid baseline for sentiment analysis.

Transformer Models (FinBERT, DistilBERT)

Provide enhanced contextual accuracy, though they are more computationally intensive.

Engagement-Weighted Sentiment

Delivers more realistic market insights by accounting for user interaction.

Integrated Analysis

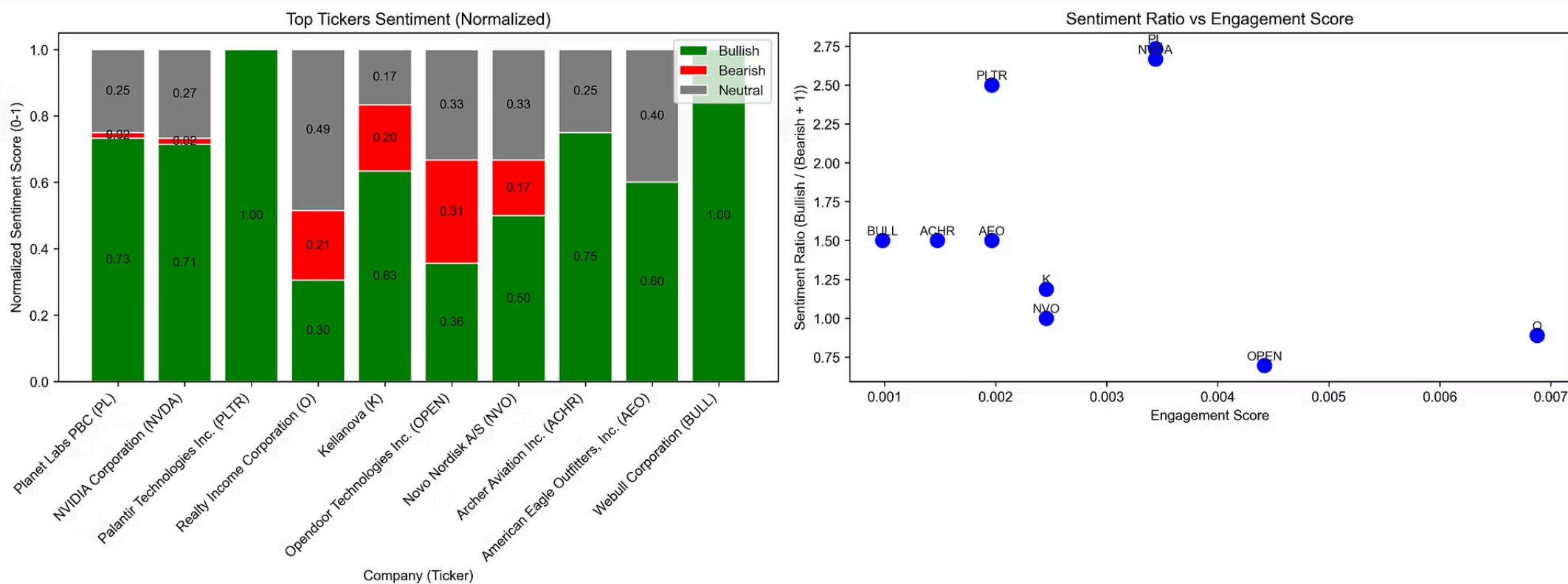
Comparing and combining these approaches generates robust investor signals.

Resources

- Github repository of project: [click here](#)
- Sentiment Analysis comparisons: [click here](#)
- BERT explained: [click here](#)
- Vader explained: [click here](#)
- TF-IDF in Sentiment Analysis: [click here](#)

Comparing results: Top 10 stocks

TF-IDF + Vaderland (6/09/25)



Comparing results: Top 10 stocks

Transformers (06/09/25)

